Object Detection with Foundation Models
Project Report

Luca Geiger

February 5, 2024

Abstract

This project focuses on the AI-based detection of wildlife in drone footage and is part of the BAMBI project. The main focus is trying out a variety of zero shot foundation models, capable of object detection inside images, to find out if these gigantic tools are useful for this task. The reason why a zero shot foundation model is so interesting for this task is, that it does not need to be trained again and will just be able to detect objects, or in this case animals, in the picture automatically.

The project itself should also provide a base for the further development of an own AI which is specified on the detection of forest animals. Currently the drone footage was taken from a lot of different wildlife parks close to Hagenberg and labeled by an animal class expert.

Although it is quite impressive that the preferred model is able to detect most wildlife correctly, a lot of research needs to be done, especially on prompting, before a certain detection can be guaranteed.

Contents

1 Introduction **4**
- 1.1 Project BAMBI . . . 4
  - 1.1.1 Goal . . . 4
  - 1.1.2 Work flow . . . 4
- 1.2 Foundation models . . . 5
  - 1.2.1 Overview . . . 5
  - 1.2.2 Zero-Shot Object Detection . . . 5

2 Setup and Model requirements **6**
- 2.1 Setup . . . 6
  - 2.1.1 Test Set . . . 6
- 2.2 Model requirements . . . 7
  - 2.2.1 Recently updated . . . 7
  - 2.2.2 Colab available . . . 7
  - 2.2.3 Well documented . . . 7

3 Test-Phase **8**
- 3.1 Reviewed Models . . . 8
- 3.2 Not working . . . 8
  - 3.2.1 Fast-SAM . . . 8
  - 3.2.2 YoloNAS . . . 8
  - 3.2.3 Yolov8 . . . 9
  - 3.2.4 Semantic Segmentation . . . 9
- 3.3 Further research necessary . . . 9
  - 3.3.1 YoloX . . . 9
  - 3.3.2 Clip . . . 9
- 3.4 Rated models . . . 9
  - 3.4.1 SAM with Transformers . . . 9
  - 3.4.2 LLaVA . . . 10
  - 3.4.3 Autodistill . . . 10
  - 3.4.4 Automated Dataset Annotation . . . 11
  - 3.4.5 Grounding Dino . . . 11
  - 3.4.6 OWL-ViT . . . 12

4 Conclusion **13**
- 4.1 Learnings and Impressions . . . 13

4.1.1 Google Colab . . . 13
4.1.2 Detection Hypothesis . . . 14
4.2 Outlook . . . 15
4.2.1 Improvement of OWL-ViT . . . 15
4.2.2 Further research . . . 16

A Supplementary Materials **17**
A.1 Introduction . . . 17
A.2 Setup and Requirements . . . 18
A.3 Test-Phase . . . 19
A.3.1 SAM with Transformers . . . 19
A.3.2 LLaVA . . . 23
A.3.3 Autodistill . . . 25
A.3.4 Automatic Dataset Annotation . . . 26
A.3.5 Grounding Dino . . . 28
A.3.6 OWL-ViT . . . 30
A.4 Conclusion . . . 33

References **35**

Chapter 1

Introduction

1.1 Project BAMBI

Figure 1.1: Logo of Project BAMBI [28]

1.1.1 Goal

Project Bambi was established with the goal to provide a yet completely new way to guarantee "an [automatic,] area-wide and accurate count of wildlife". [28]

1.1.2 Work flow

Drone footage

Big drones are used to capture both thermal and zoomed video footage of an area, to later detect animals in it. Because most animals live in the woods, at least in the currently checked region, trees can make it really hard to detect animals, especially from an eagle perspective.

Light-field rendering

However with a new light-field rendering technique, which basically takes a set amount of images inside the video into account and remaps them together for a sharpened bigger image, animals are easier to detect. But this not only includes sharpening, but also the possibility to change the height at which the images are blended together. In this case the forest floor, so ca. two meter above the ground is the zone of interest, and thus completely getting rid of the trees above the animal.

Classification

After this is processed, the newly created images should be automatically classified by an AI, using object detection methods. Since there are a lot of AIs available, with different purposes and strengths, a research on this topic was needed, which this semester project was used for.

The most similar project found on the research, is located in Germany, run by the Wild Intelligence Lab and focuses on the detection of the wildlife in Namibia, mainly to support its conservation. [14]

1.2 Foundation models

1.2.1 Overview

Nowadays object detection is extremely good, even able to detect objects behind other objects in difficult conditions (such as a busy intersection). But they have constraints: First of all they are extremely expensive to train and the training needs a big amount of time, even with the best equipment available today. But what makes it even worse is the fact, that they are mostly built for a specific use case (such as the detection of vehicles), so their accuracy varies drastically depending on the provided image.

1.2.2 Zero-Shot Object Detection

This is where Zero-Shot Object Detection comes into mind. “Zero-shot classification models are large, pre-trained models that can classify images without being trained on a particular use case”. [10] This would allow BAMBI to completely extinct the critical problem of the specific use case, while maintaining the good results. It further supports an easy way to create an own AI, specifically designed to detect the wildlife properly, using the zero-shot model as foundation.

Chapter 2

Setup and Model requirements

2.1 Setup

The provided footage includes 60 minutes of video material in two different conditions: snowy and clear weather. Furthermore this footage clearly contained animals since it was captured from nearby animal parks and zoos.

2.1.1 Test Set

To start with the testing of the AI, a proper test set is necessary, which in total, contains 24 images and 10 short videos extracted from the material by VLC-Player. [15]
This is of course not enough to be a test-set for a proper evaluation, but good enough to gain first impressions on how well the researched model is suited for the task.
Of course the test-set was not created without any thoughts, but instead categorized as accurate as possible.

Categories

The first category is based on the weather. It is either clear or snowy, and needed to see, if the weather has any influence on the detection.

The next category is focused on the estimated difficulty (easy, mid, hard) of the detection. It was created, because the detection of animals/groups was estimated, to be much easier when no trees are between them and the drone. A.2, A.3

The last tested category is based on the number of animal species visible in the same image. It can either be one or multiple, to find out if an AI is able to classify the different species correctly.

There were also other categories at first such as birds, small and big animals and almost impossible to detect ones, but the time did not allow to research them further, so they got suspended.

2.2 Model requirements

Since there are numerous models available, for both object detection in images and videos, it was mandatory to discuss requirements, especially since some models are only usable when setting them up oneself.
In the beginning, this was done with three different models using docker containers, but it soon got clear that the time spent for the setup, was too much to gain just an overview.

The models also have to fulfill three criteria, which are necessary for BAMBI to implement them into their workflow easily. The models need to deliver good results, have a low amount of user interaction and should be working automatically (or be rewritten to work automatically without much effort).

The requirements for the provided example-version to be reviewed, are another three requirements, needed to be met for a quick impression.

2.2.1 Recently updated

The fact that new and better models are published rapidly, only allows identifying the currently best suited model. To prevent researching an old model, one of the requirements that needs to be fulfilled, by both the model and the example, is a frequently done update.
This also implicates, that the model keeps being improved and is supported well by their publishers.

2.2.2 Colab available

Since this project mainly targets the quick test of models, Google Colab [11] is one of the best ways to gain impressions fast.
If a Colab version is already available containing the model, it can also be suggested, that it is refactored well, which makes it easier to access and understand the underlying system.

2.2.3 Well documented

Last but not least it is very handy to be able to find some quick support, if something is not working properly. It also proves the reliability of the model and enables a further look into the background, such as files published on GitHub.

Chapter 3

Test-Phase

3.1 Reviewed Models

Overall 13 models were tested, but only six are further mentioned here. The results of the others are either none at all, or the effort needed to get the model working was too high, at least since this project focused only on consistently working models. [29], [24], [12]

3.2 Not working

3.2.1 Fast-SAM

Fast-SAM [3] is developed by Meta [20], and is able to mask an image based on a user set input. This includes either coordinates for a point or a box, which would not be useful for our task, but also a prompt, which then creates a proper mask, that could manage the object detection well.

Even though the model looked promising it failed with an internal error, which needs to be fixed in a module itself, consuming too much time to investigate further.
The exact error is the following: "AttributeError: 'str' object has no attribute 'shape'".

3.2.2 YoloNAS

Yolo NAS [26] is developed by Deci AI [9] and is described as the best working AI based on the average mAP on Roboflow-100 compared to other Yolo models. [26]

It runs trough the training process using the self-built Roboflow [13] model, but then it fails because of an incorrect checkpoint path, before the final evaluation. Since this is handled internally, it is not clear where to start the research and thus how much time a fix needs, so this model was abandoned.
The printed error message says the following: "FileNotFoundError: Incorrect Checkpoint path: /content/checkpoints/semester_project/average_model.pth (This should be an absolute path)"

3.2.3 Yolov8

As mentioned before Yolo NAS [26] was developed after Yolov8 [22], but it still got reviewed due to it´s pioneering breakthroughs in 2023.

But even after several tries the model crashed at the same line of code again, with the following error: “FileNotFoundError: [Errno 2] No such file or directory: ’/content/runs/detect/train/confusionmatrix.png’”

3.2.4 Semantic Segmentation

This is the only model that worked [not cited since it got removed], but is listed here because it did not provide a useful result for the task. There needs to be many inputs set, including colors and labels matching each other, which would not be the problem, but the imported example is a notebook itself, so it is totally unclear how to change it to an own example without much effort.

3.3 Further research necessary

3.3.1 YoloX

The example for this model showed the detection of malicious cells in an X-Ray image, which supposedly allows to detect shapes in thermal images as well. [27] However the Colab was not offering any results, so it got suspended quickly. This might have been to early when reviewed from today’s perspective, especially since it was accidentally not researched any deeper afterwards because it fell out of focus.

3.3.2 Clip

Clip [25] is a tool offered by OpenAI [1] and is especially good for image detection. The example needs a custom roboflow model, which would have been available already, but it also needs a list of classes, to then calculate it´s correct prediction percentage. These classes need to be set by hand in a text file which was not manageable quickly, so the model got suspended. Since it was just found a few days before the final presentations it would thus not have had the same amount of research time as the others which makes the rating unfair.

3.4 Rated models

To provide a better overview, the models were rated by the before mentioned three criterias. However these categories are not based on any scientific data, but instead on the impressions gained meanwhile the research, which are subjective.

3.4.1 SAM with Transformers

SAM stands for Segment Anything Model [21] and is developed by Meta [20]. To fulfill the task better, a Colab was set up where are also transformers included. This Model

needs an Image as an input which will then be masked based on points and/or box coordinates set by the user. [30]

For points the results were not useful, but with the box approach the masks were very accurate. But concerning the criterias this model is not useful for the automated detection of animals, because even though it delivers good results, the amount of user interaction is very high. This is mainly because of the here used coordinate approach, which makes automating nearly impossible and also works only for one image at a time. A.4, A.5, A.6, A.7, A.8, A.9, A.10

3.4.2 LLaVA

LLaVA [16] is a chatting model, which needs an image input to answer questions regarding it. So here prompting is extremely important and the results vary widely depending on it. With very little information, the model is not able to detect an animal at all. A.12 If it knows that an animal is present, it was able to correctly count them. A.13 And when a huge amount of information is given, the answer can be very useful, even though it is not clear if it just mostly repeated the input. A.14 The model sadly also does not fit our requirements, especially since automation is a big task and the user needs to set a specific input prompt to make the model work.

3.4.3 Autodistill

Online

Autodistill [5] automatically converts videos into images based on a user specific amount of frames (e.g. 10 means every 10th frame an image is extracted from the video). It furthermore is capable of detecting the objects inside the video, by a set prompt and automatically marks them with a box and a probability.

Sadly this takes a lot of RAM and since the free version of Colab only offers 12GB, which got exceeded every time with the test videos (even one alone), there is only one video to rate. This video shows that the detection is useless at least for this clip, since it got marked completely as the provided prompt. A.15

Local

Because of the RAM issues a bigger graphics card was needed, which happens to be a DGX A100 [8] located at the university itself. This offers not only 40GB of RAM, but also the connection via SSH [17], which allows to connect to it remotely. However there were a few problems which needed to be overcome. First of all the SSH-Key got corrupted somehow, which was fixed by creating and implementing a new one.

Afterwards the SSH key worked, but the docker container which was created to open up a Jupyter Notebook Server (necessary to run the Colab locally) was not reachable in the browser. The problem was that even though the container port was correctly set (▷ docker run –gpus '"device=7"' -p 8888:8888 –name CustomColabServer lucag97/colab-server), the SSH connection needed to include the same port to ensure the connection,

which can be done using the following command: ▷ ssh -L 8888:localhost:8888 -J jump-dgx-a100@193.170.124.50 username@193.170.124.53.
Connecting the Colab to it, is as easy as clicking on "Connect to local runtime" and entering the token provided from the docker container there (contains localhost not tree!).

Now the connection was working, a few other problems occurred. In the beginning the Colab did not work at all because a root user is not allowed to use "pip install", since this would impact critical system data. To solve this issue, a docker file was created which included the setup for a non-root user and got published as a docker image in the hub. [19]

Now the installation via pip was fine, but the modules seem to not being loaded correctly, so the easiest solution was to already install them in the docker file. After this was done the Colab ran without any problems until it needed the user specific dataset which was provided by the implementation of Google Drive before. Since this method can not be used in a local Colab, an own Roboflow model [18] needed to be created to provide the dataset. This also worked immediately after the creation, but the Colab still needed to be suspended because of now visible internal issues inside the Colab, causing an automatic restart without further information.

Overall result

To provide a rating here would be unfair, but the model still might have potential because of its ideal workflow, especially considering the automatic conversion of videos into images. Furthermore prompting could make a huge difference, but since the model was not working in time, this could have been tested properly yet.

3.4.4 Automated Dataset Annotation

The automated dataset annotation is done by a Colab using Grounded SAM (a mixture of Grounding Dino and SAM). [6] So image(s) are needed as input, followed by prompt(s), which than produces masked image(s) with probabilities.

The results are overall not pretty good, but it creates very accurate masks sometimes, which might be supported by better prompting. A.16, A.17, A.18

3.4.5 Grounding Dino

Grounding Dino [7] on its own needs also an image input, as well as a prompt, but it creates boxes around the detected objects instead of masks and also marks the probabilities.

Overall the results are pretty good, and together with the low user input and the given easy automatability, it could be a good solution to get the task done. Nevertheless there are too many boxes, which could be fixed by better prompting and very low thresholds are needed to get good results, which can become problematic, regarding the number of boxes. A.19, A.20, A.21, A.22

3.4.6 OWL-ViT

The detection of OWL-ViT [2] can be done in two ways at the same time: Text-conditioned and Image-to-Image. It only needs images as an input to detect the objects and draws boxes with probability notes around them on the fly, based on the user input.

Text-conditioned

Here a prompt is used to ensure the detection. The detection is then done character by character, so the user can see what change the current character makes for the detection. The results are good, but could be even better by researching the prompting. A.23

Image-to-Image

The user must draw a box here by using a boxing tool, which makes the interaction extremely fast. OWL-ViT itself then creates its own box based on the user input, which afterwards is used for the detection inside another image. This works even in hard conditions such as animals barely sticking out of the tree, but of course there are still too many detections inside the image. A.24, A.25, A.26, A.27

Overall results

This model is by far the best, especially when using Image-to-Image detection. The automation of the model should be as simple, as instead of only using the detection for one picture, providing an array, to make the detection for all pictures at once. The only problem is, that the Colab only works every second to tenth restart, which was not further researched yet, since it works either consistently or not at all.

Chapter 4

Conclusion

Zero-Shot Foundation Models are capable of fulfilling the task, but it needs a lot of improvement to reach the set goal by BAMBI.

4.1 Learnings and Impressions

4.1.1 Google Colab

Usability

To expect a Colab to work immediately after the branching is more a wish than reality. Only in three out of thirteen cases, the Colab worked without major changes, when implementing the Roboflow model or the test-set itself. Sometimes even the unchanged version of the provided Colab doesn´t work properly, which makes it even more difficult to get an impression quickly.

Errors

Most of the errors faced were solvable until everything on the "surface" was fixed and they became located deeper. This means everything from not implemented modules, which needed to be researched first to tell where the problem comes from, up to internal issues, like incorrect building of the weight matrix. To fix these errors, one must search fairly deep in the program, which for this project was not done, due to the goal of gaining fast impressions on working notebooks.

Local usage

Running a Colab locally causes a lot of not expected issues, like user dependencies or no possibility to provide the test-set every time again, as before done by implementing Google Drive. It also seems to cause problems with the downloaded modules, even though nothing is changed compared to the original Colab. Thus the local usage of Colab should only be done when no other option can be found and enough time is available to research errors.

Overall impression

Google Colab is an amazing tool to gain first impressions of almost all models when it works correctly. Nevertheless it can sometimes be very tricky to fix an error, due to the little knowledge a user gains while using the notebook. To fix the deeper laying errors, it is mandatory to have a good documentation at hand, that allows oneself to fix the problems properly.

4.1.2 Detection Hypothesis

Sun / Shadows

Sun and shadows can cause problems with the identification of the object, since sunny parts inside the thermal image are also white, which is the same with animals. This makes it even for a human hard to decide, if it actually is an animal or just a sunspot and can only be identified, when seeing the whole sequence. A.28

It would be easier to identify it correctly, if the AI would know the context and understands that the two images, are the same alongside each other, because most of the time, the zoomed part allows a clear classification.

Population

The population inside an image makes a huge difference. The more is visible in the image, the harder is the correct detection. This especially is the case for wooden areas, such as branches and tree stumps, since it is almost impossible to make a clear statement, whether there are animals underneath or not. A.29

This should become way better when using the lightfield-rendering before the actual detection. The zoomed part also does not really help in this case, since it does not allow to look through the trees at all.

Results

All of the AIs are promising better results than the ones achieved in this project. This could be the case, because all of the examples for the models use a non-eagle-perspective to show the detection whereas this project focuses on the eagle perspective only. The reason this is suggested, is that in most of the images, the detector also marked parts of the image, that look like an animal from the a non-eagle-perspective. It furthermore had higher probabilities combined with these boxes, which can be seen as an indicator for a reference image using another perspective. A.30

A simple google search with the prompt “animal” showed, that only almost every 100th picture, is clearly captured from an eagle perspective. If the model was trained using google images, which again comes down to the black box problem, it can hardly provide as good results on eagle perspective images, compared to non-eagle ones.

Overall impression

This directly leads to the fact, that prompting is by far the most important part to research further. Especially for the eagle perspective it seems quite mandatory to test this better, due to it´s uncommon appearance in the images (Google) when entering the same prompt. A.31

It also does not make any difference when changing the scale, the resolution or the sharpness of an image, except that it can cause Windows to shut down showing a blue screen without any warning (even though it worked before). The models were all tested using the same 10 prompts, which is not enough to tell if a model works good, but it allows a fair testing process.

4.2 Outlook

4.2.1 Improvement of OWL-ViT

Detection inside populated images

The next steps that are needed to be taken, include especially the improvement of the detection inside populated images. For this, it is mandatory to use images which already took place in the lightfield-rendering process, to eliminate the wooden areas as far as possible.

Prompts and reference settings

The biggest problem then is to extinct the wrongly set up boxes by either tweaking the prompts or other values such as the threshold. Here are only to values possible, namely the MIN-CONFIDENCE which should be as high as possible and the NMS-THRESHOLD which influences the number of boxes, so it should be as small as possible. The tricky part however will be to find the best values, while still keeping the good results.

Automate tests and collect data

It could be helpful to find suitable values first and then testing around close to them with an automated test. In this case, this needs to be implemented, by changing the image, to compare with an array of images and then storing the results, to evaluate them later.

This should be done with scientific methods, namely by measuring the accuracy and other necessary parameters, to provide more than a subjective result.

4.2.2 Further research

Test of other Models

Since models are published frequently, there will also be other models to review along the improvement path. One model that might be promising is the Set-of-Marks-model [23], which creates a mask to than use it with GPT4. This allows to ask questions based on the mask and could lead to a faster learning process, but also gaining more insights on their predictions.

Building an own model for the task

In the end an own model needs to be built for this task, to both ensure a guaranteed and correct detection of the wildlife in the images. This research only provided a base knowledge, to see whether the models are capable of detecting animals or not. It also suggests that OWL-ViT is the best for the task, based on the tested models and that it could be used as a foundation for the self-built model.

Appendix A

Supplementary Materials

A.1 Introduction

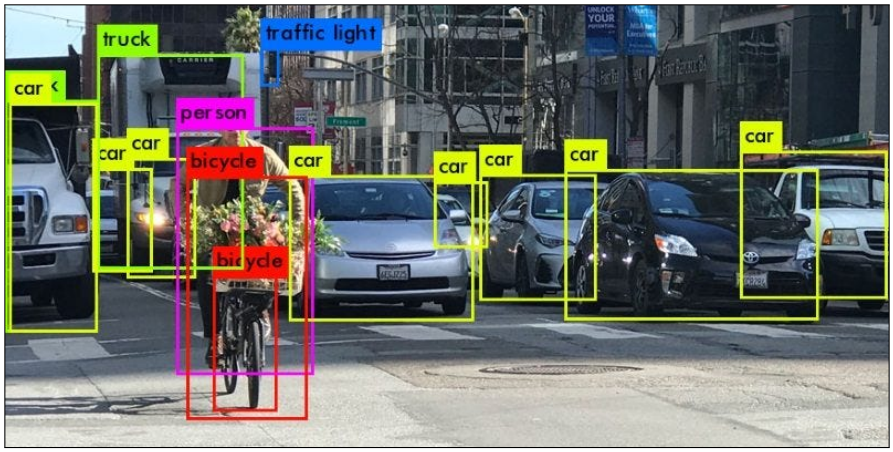

Figure A.1: Reference picture for the capabilities of todays models featuring an advanced Yolov7 Model for automated driving [4]

A.2 Setup and Requirements

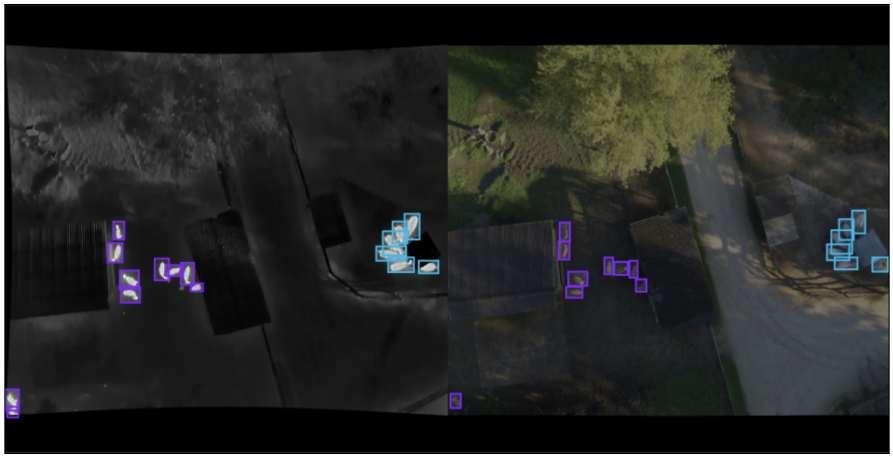

Figure A.2: Test set example fulfilling the categories: Easy, Clear and Multiple Classes

Figure A.3: Test set example fulfilling the categories: Hard, Snowy and Only One Class

A.3 Test-Phase

A.3.1 SAM with Transformers

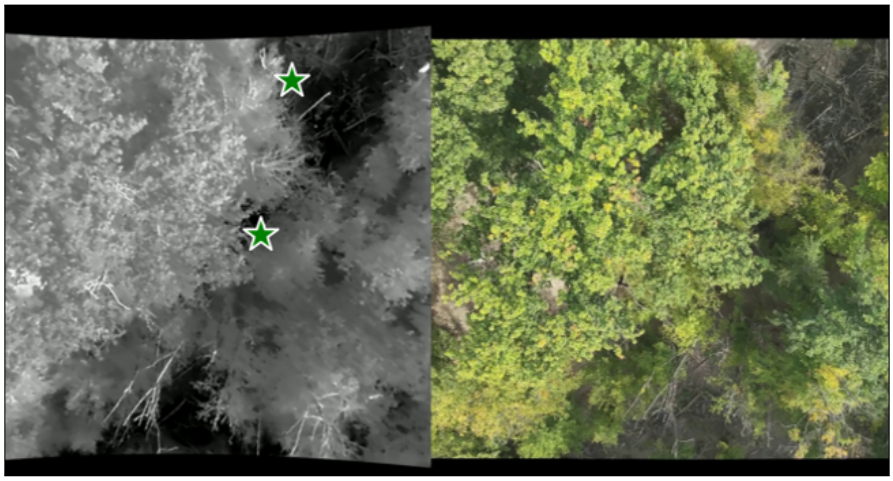

Figure A.4: The input used for the point based masking. The animals are under the stars which are located by x-y-coordinates

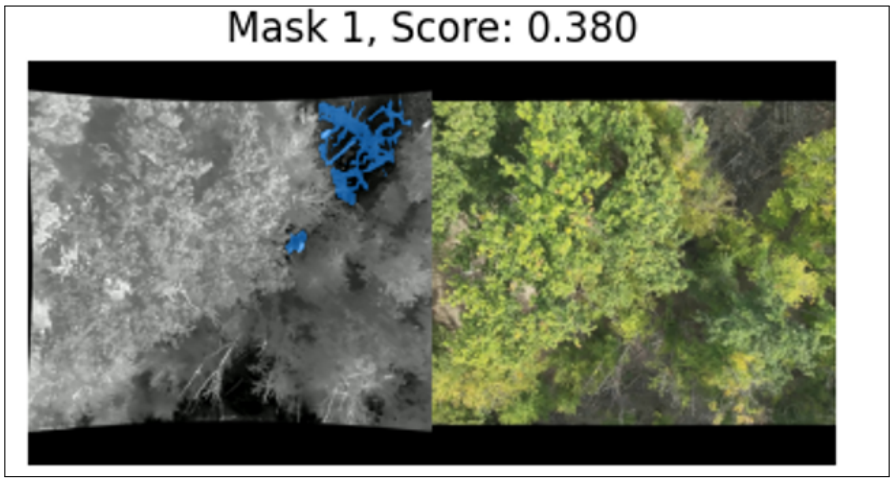

Figure A.5: First mask created based on the points

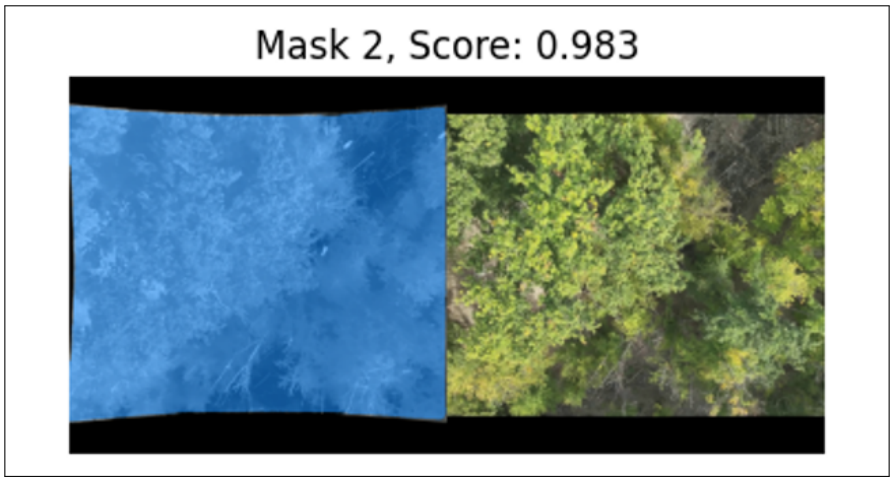

Figure A.6: Second Mask created based on the points

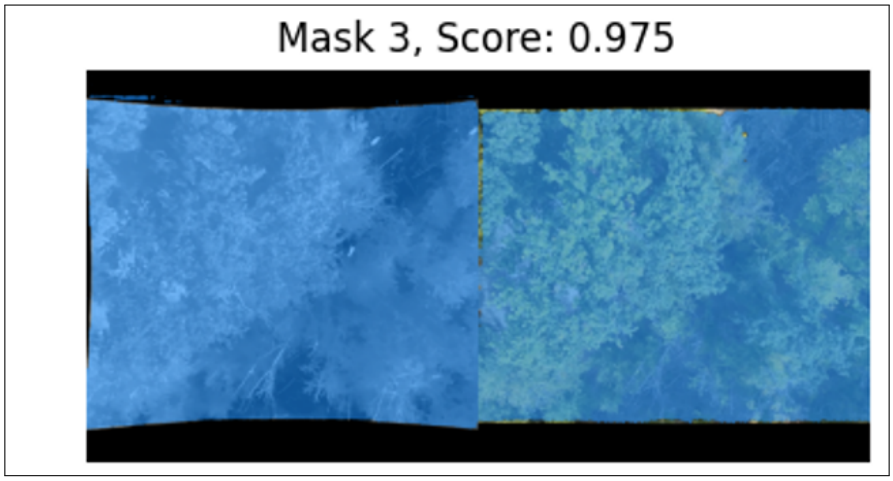

Figure A.7: Third mask created based on the points

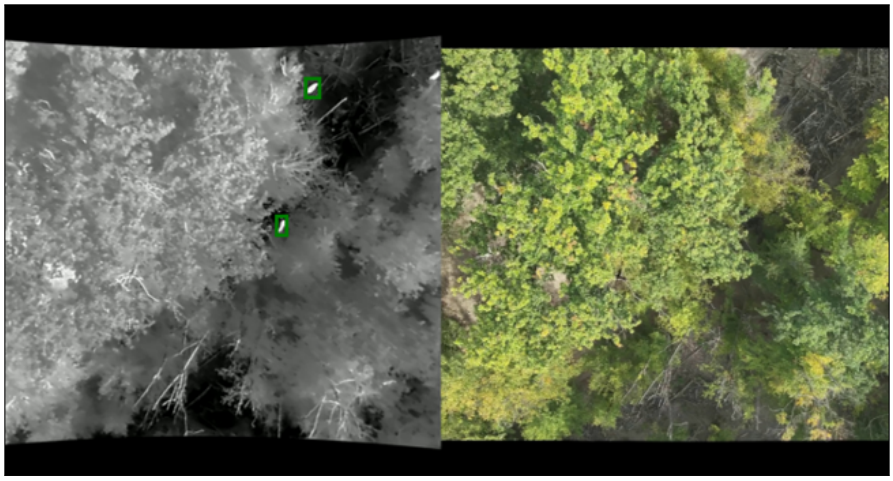

Figure A.8: Input for the box based detection. Here the upper left and lower right coordinates are needed for a proper input

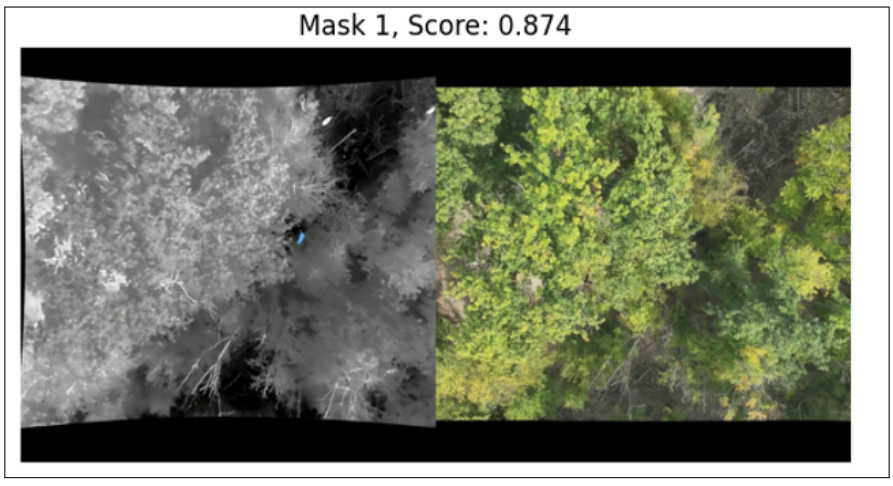

Figure A.9: First masked created from the box input

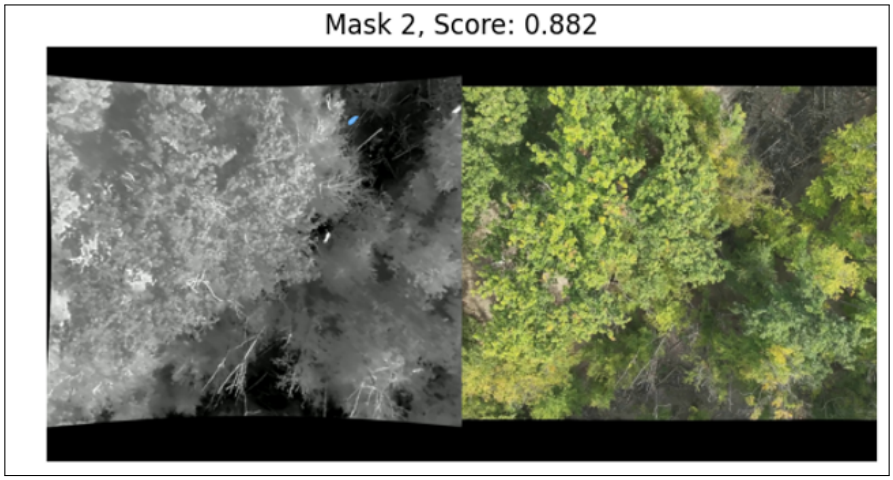

Figure A.10: Second Mask created from the box input

A.3.2 LLaVA

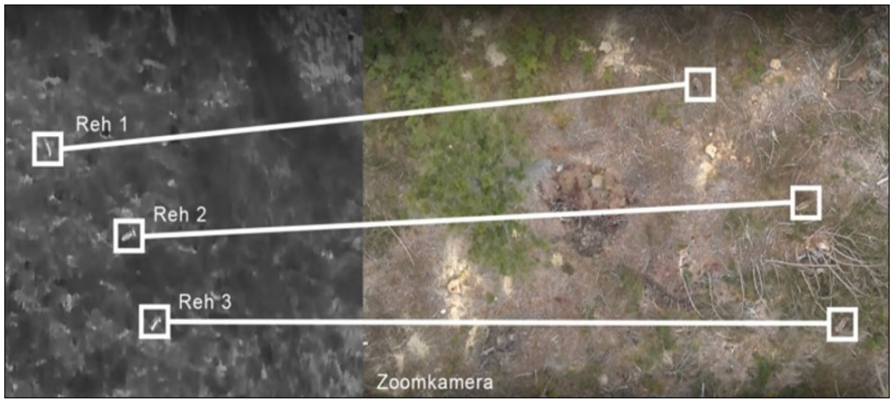

Figure A.11: This image was provided as the teaser image for the semester project and used as an input for LLaVA

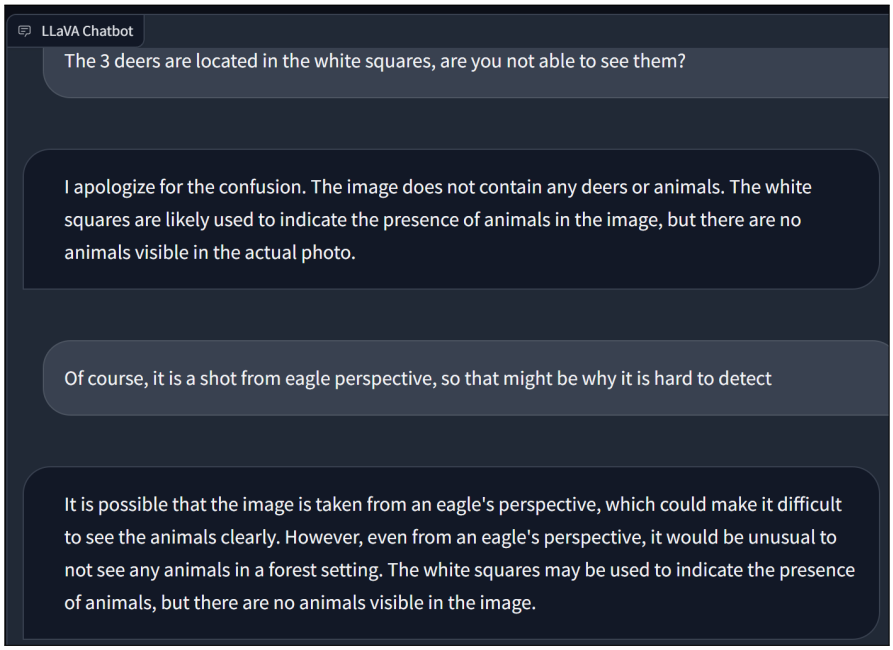

Figure A.12: The first test was with very little information, just the question: Are there any animals in the image? It soon became clear that the model is not capable to provide a good answer without more information, which then resulted in this output

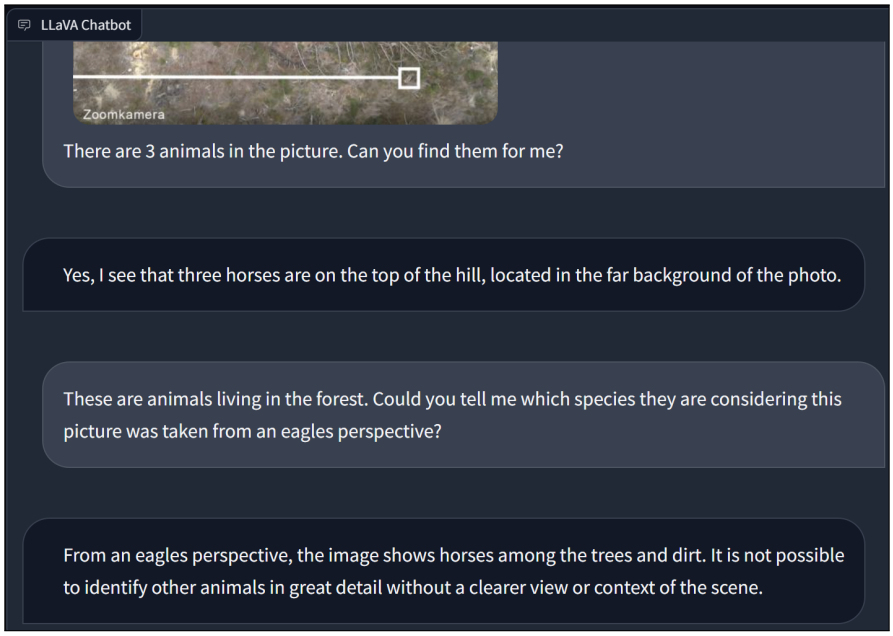

Figure A.13: For the second time more information was given, to check whether the model is able to locate them. Sadly the model is not able to output coordinates

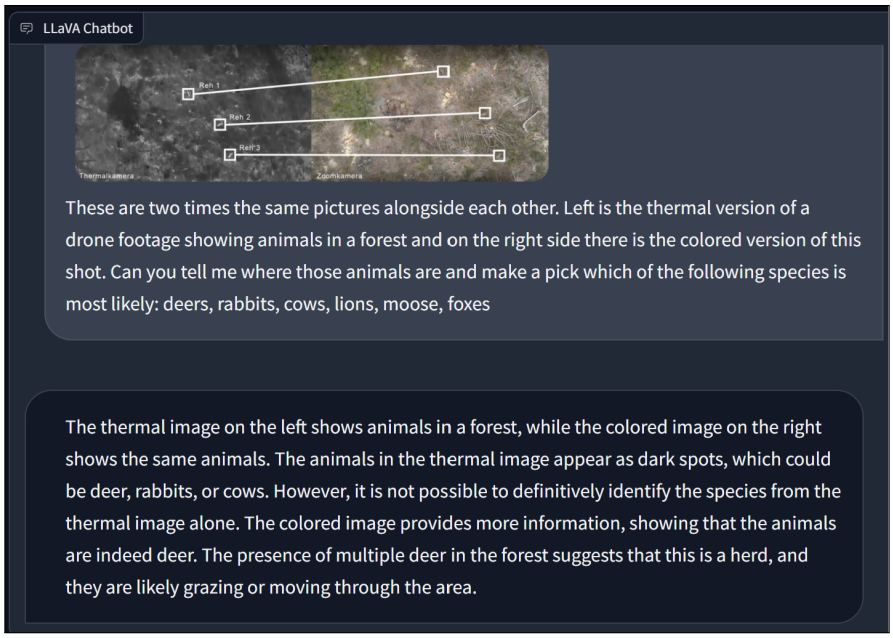

Figure A.14: For the final test the model received as much information as possible which lead to quite a good output. The model seems to understand the connection between the two images and also gives additional information to it

A.3.3 Autodistill

Figure A.15: This is a screenshot from the video, showing that the animal located under the tree is not properly detected

A.3.4 Automatic Dataset Annotation

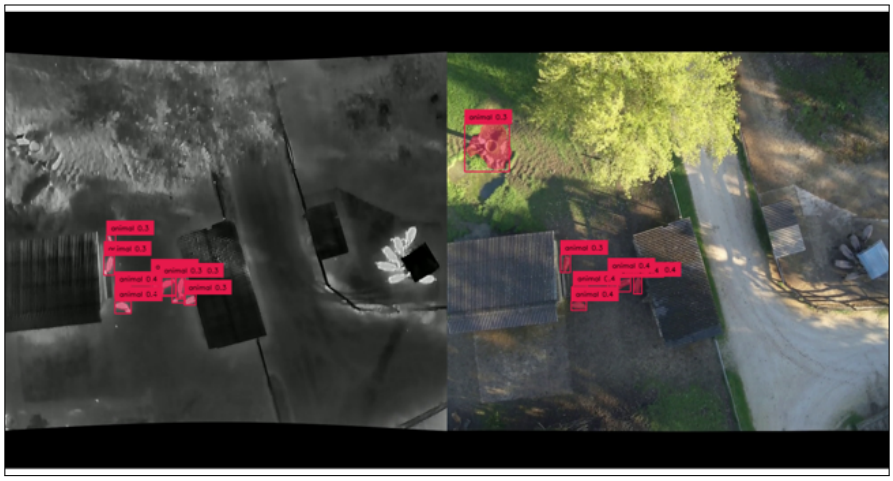

Figure A.16: This is one of the 2/24 good masks, where everything got correctly masked, except for the well and the Tarpans on the right of the image

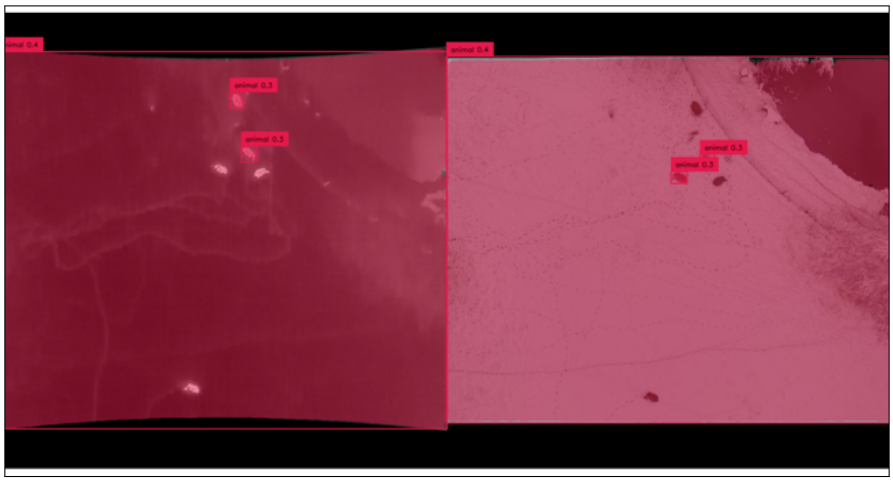

Figure A.17: This is one of 2/24 pictures from the mid category based on the mask. The model detected some of the animals correctly, but also tended to mark the whole image as an animal

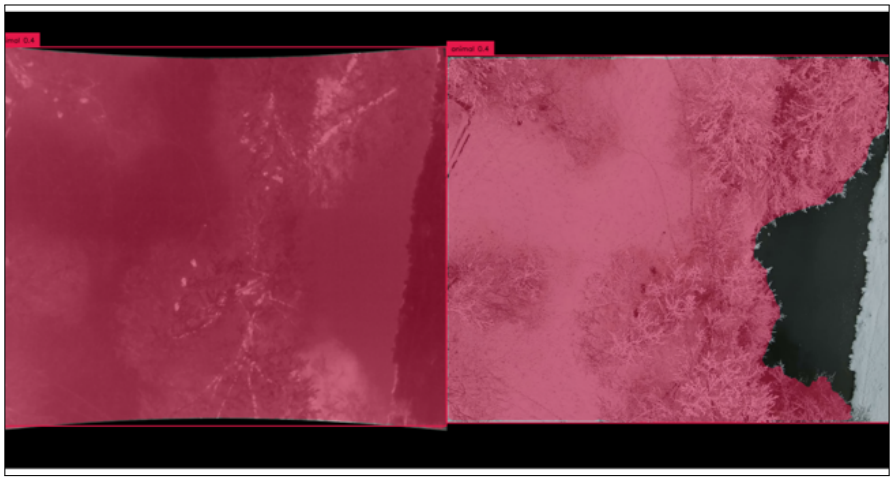

Figure A.18: One example of the 20/24 images which were badly detected. All of them were (almost) completely marked

A.3.5 Grounding Dino

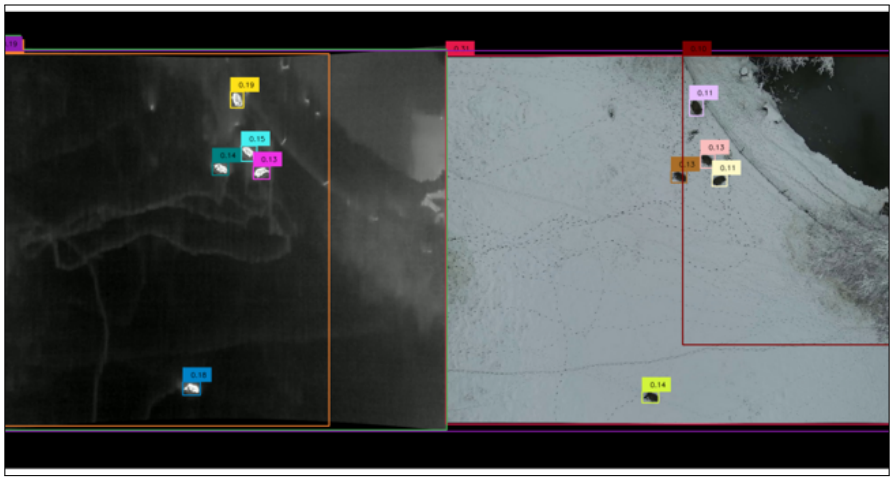

Figure A.19: This was the result when prompting "sheep". The detection only worked for all animals when using a low threshold (<= 0.11)

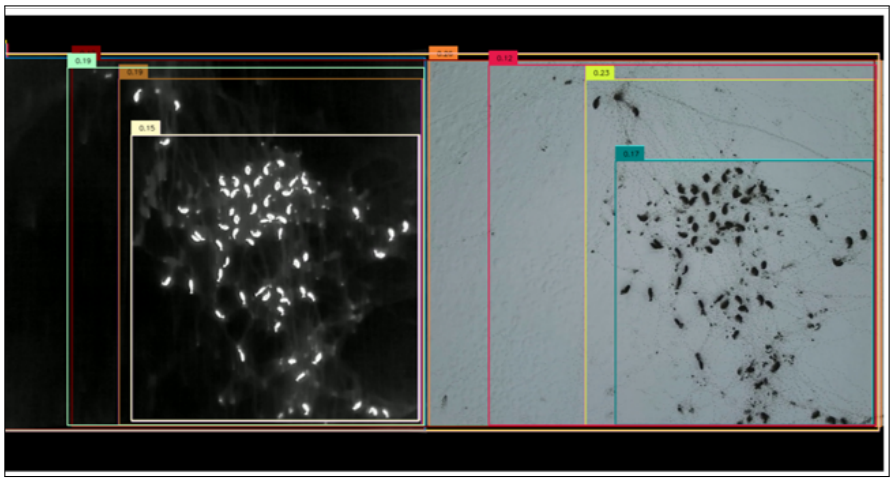

Figure A.20: The "flock of sheep" got also detected accurately. However it is tough to tell how good the detection actually is based on this example

Figure A.21: Even with multiple classes the model detects almost all animals. It still marks to many objects inside the image such as the well here

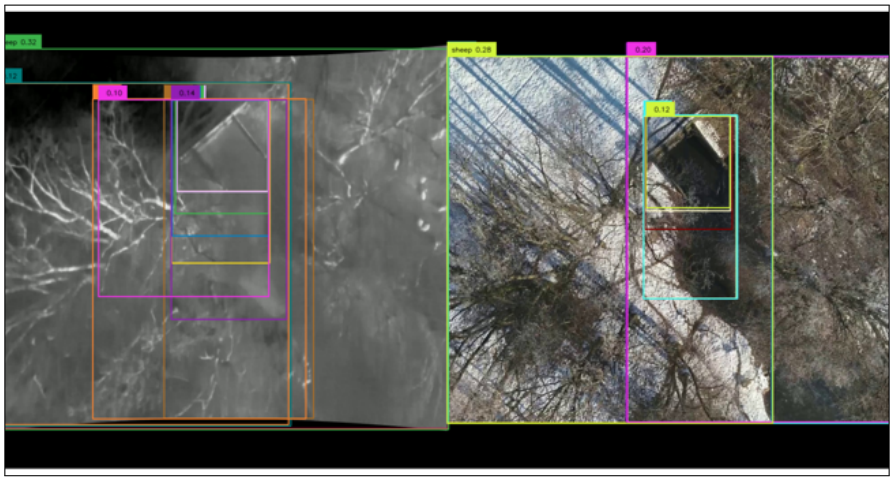

Figure A.22: This image shows animals on the right of the image. However the model did detect the bridge instead

A.3.6 OWL-ViT

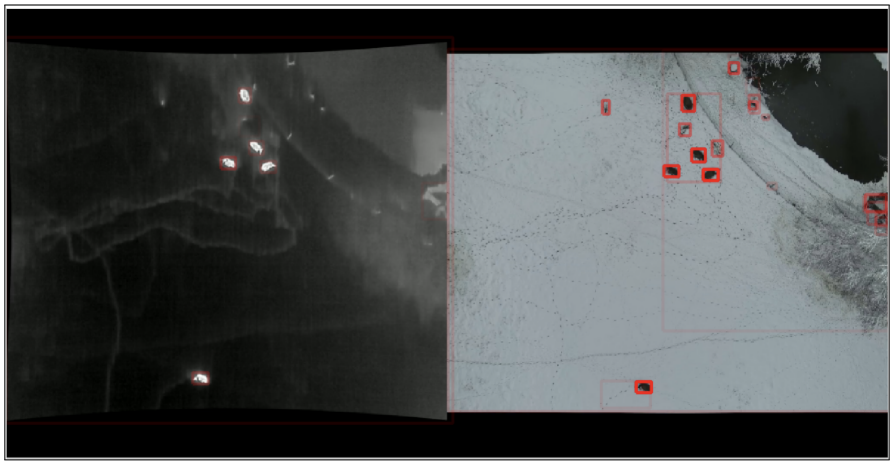

Figure A.23: The text conditioned detection of OWL-ViT using the prompt “deer shape”. All animals are correctly detected here, and seem to have received the biggest mark as well

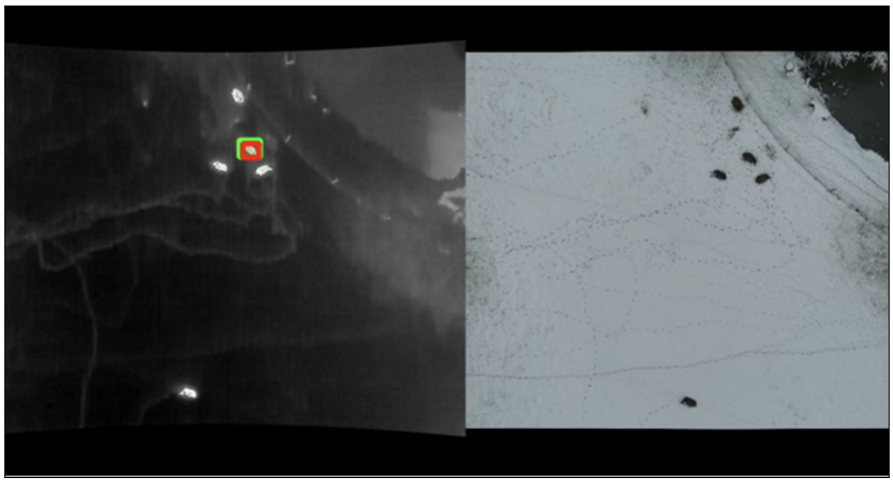

Figure A.24: The input for Image-to-Image-detection is done by a user set box which is the green one here. The model than builds it´s own box based on this input colored in red and used for the detection

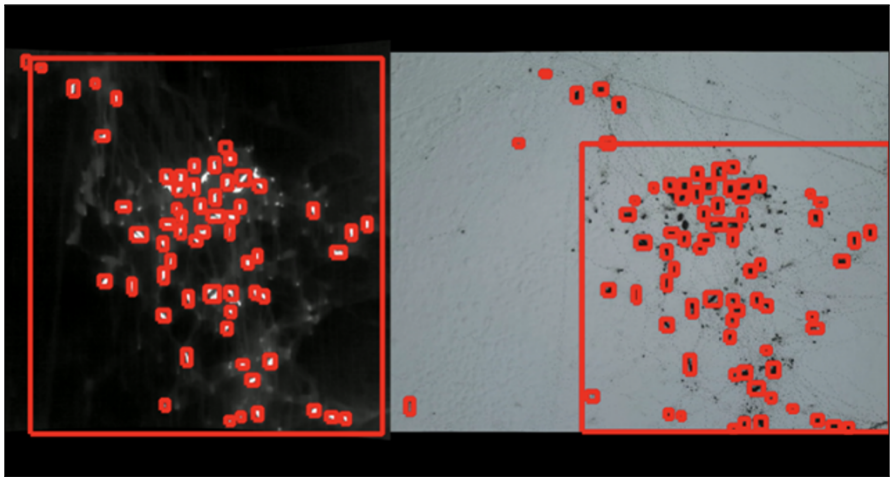

Figure A.25: Almost every animal in the group was detected in an easy rated image. There are still too many detections, but it is already quite accurate

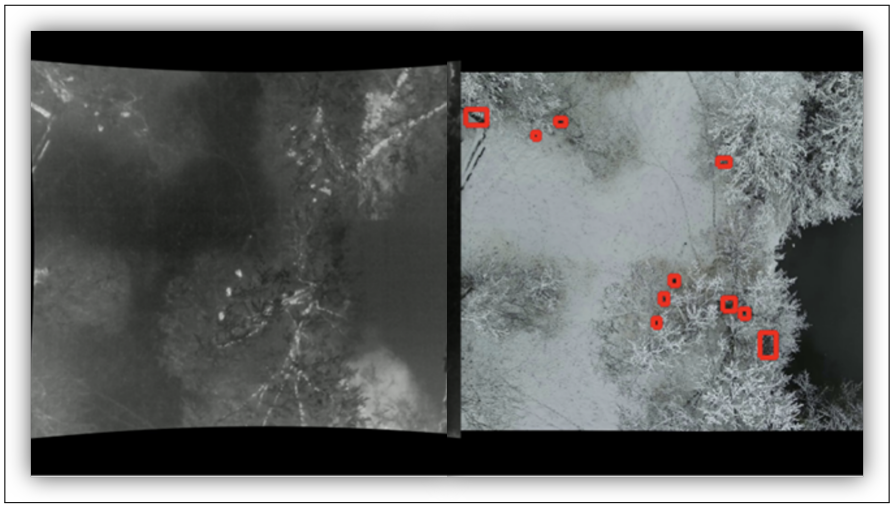

Figure A.26: The image for the mid category also got detected correctly. Even the two sheeps located on the top left here are correctly marked

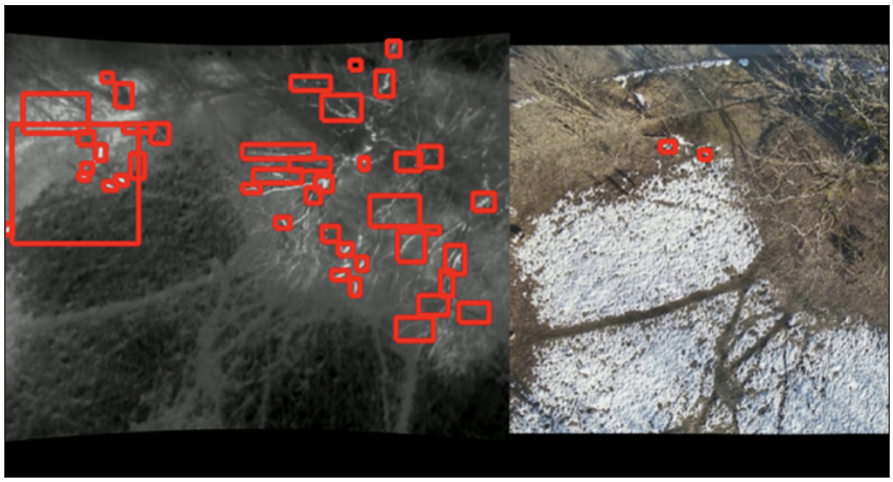

Figure A.27: The hard rated image is not as accurate as the two before but it still offers a good detection. However here are way too many boxes and it missed a few animals completely

A.4 Conclusion

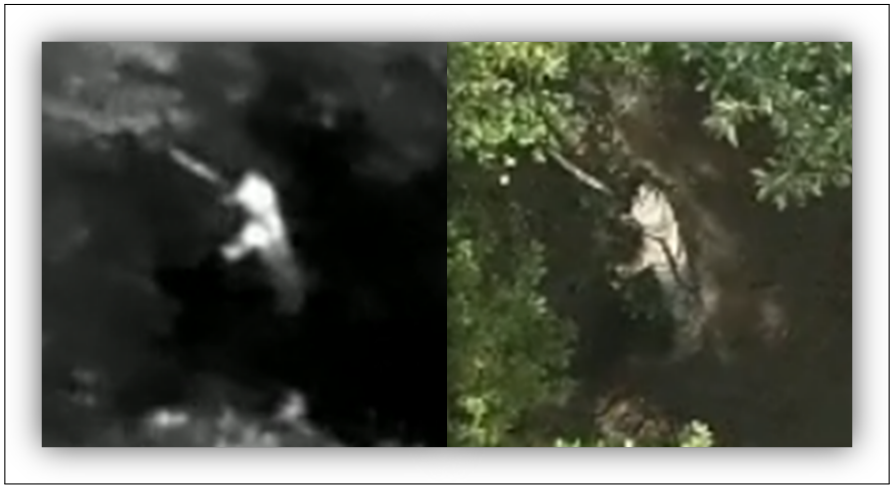

Figure A.28: This is a zoomed in screenshot of one of the clear videos. With only knowing the thermal image, it is very likely to estimate an animal to be visible there. But with the zoom footage, it is more likely to be a tree stump instead

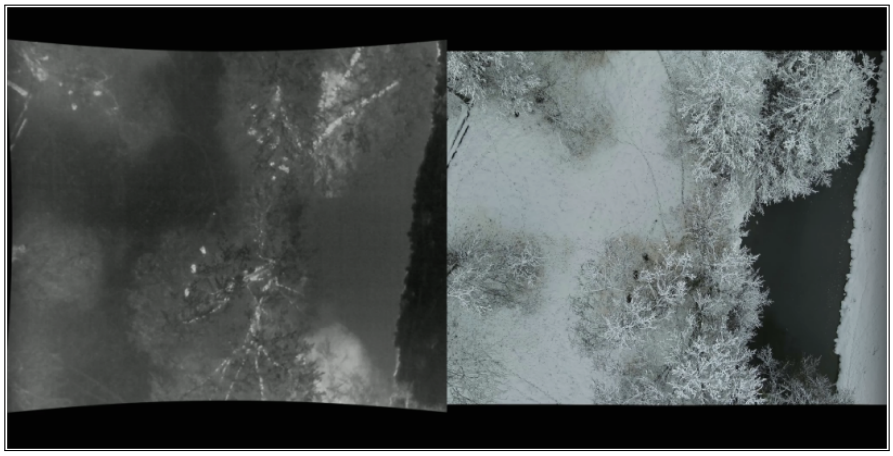

Figure A.29: Wood also appears as bright spots in the image, which makes a detection tough. When the animals are under a lot of branches, they might be missed by the detection algorithm

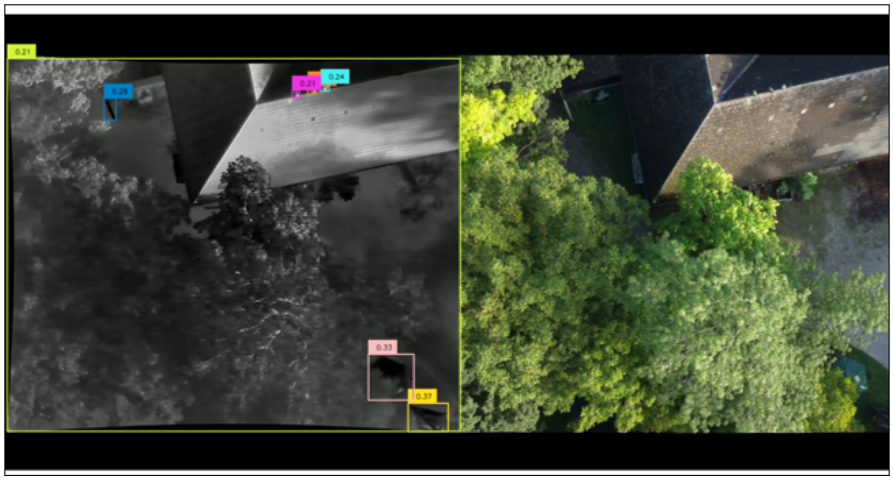

Figure A.30: This was the image that suggested that the model tries to detect from the front perspective. In the bottom right the highest probability can be found around two shadows actually looking like birds from a far. The prompt was “bird thermal perspective” here

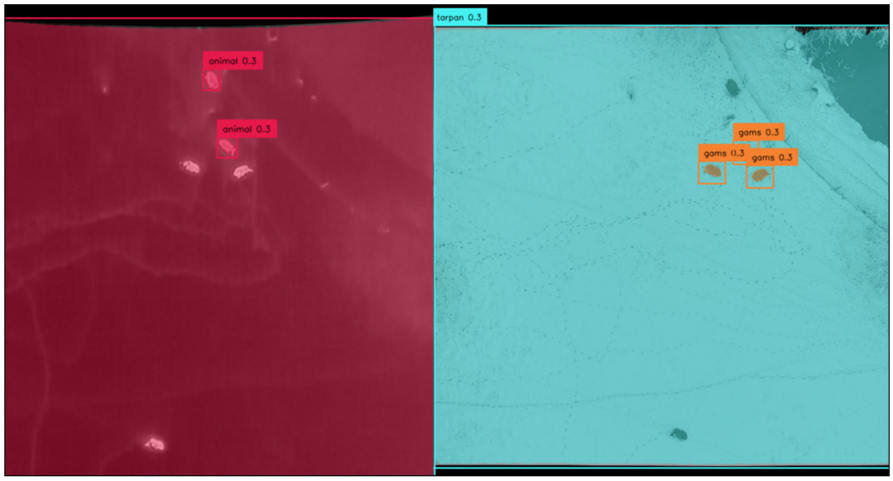

Figure A.31: This image shows the difference between two different prompts. It needs to be researched whether a more specific prompt is better suited

References

[1] Open AI. *OpenAI-Homepage*. 2024. URL: https://openai.com/ (visited on 01/31/2024) (cit. on p. 9).

[2] Scenic Authors. *Scenic-OWL-ViT*. URL: https://github.com/google-research/scenic/tree/main/scenic/projects/owl_vit (visited on 01/31/2024) (cit. on p. 12).

[3] berry-ding. *CASIA-IVA-Lab/FastSAM*. June 4, 2023. URL: https://github.com/CASIA-IVA-Lab/FastSAM (visited on 01/31/2024) (cit. on p. 8).

[4] Jayita Bhattacharyya. *Building an Advanced Object Detection Application for Autonomous Vehicles: YOLOv7, Intel PyTorch*. June 22, 2023. URL: https://itsjb13.medium.com/building-an-advanced-object-detection-application-for-autonomous-vehicles-yolov7-intel-pytorch-478ee5cedd39 (visited on 01/31/2024) (cit. on p. 17).

[5] capjamesg. *Roboflow-Autodistill*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/how-to-auto-train-yolov8-model-with-autodistill.ipynb (visited on 01/31/2024) (cit. on p. 10).

[6] capjamesg. *Roboflow-Automated-Dataset-Annotation*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/automated-dataset-annotation-and-evaluation-with-grounding-dino-and-sam.ipynb (visited on 01/31/2024) (cit. on p. 11).

[7] capjamesg. *Roboflow-Grounding-Dino*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/zero-shot-object-detection-with-grounding-dino.ipynb (visited on 01/31/2024) (cit. on p. 11).

[8] NVIDIA Corporation. *DGX-A100-Homepage*. 2024. URL: https://www.nvidia.com/en-us/data-center/dgx-a100/ (visited on 01/31/2024) (cit. on p. 10).

[9] Deci. *DeciAI-Homepage*. 2024. URL: https://deci.ai/ (visited on 01/31/2024) (cit. on p. 8).

[10] James Gallagher. *What is Zero-Shot Classification?* Nov. 16, 2023. URL: https://blog.roboflow.com/what-is-zero-shot-classification/ (visited on 01/31/2024) (cit. on p. 5).

[11] Google. *GoogleColab-Homepage*. URL: https://colab.google/ (visited on 01/31/2024) (cit. on p. 7).

[12] HuggingFace. *Lists of HuggingFace-Models*. URL: https://huggingface.co/docs/transformers/model_doc/sam (visited on 01/31/2024) (cit. on p. 8).

[13] Roboflow Inc. *Roboflow-Homepage*. 2023. URL: https://roboflow.com/ (visited on 01/31/2024) (cit. on p. 8).

[14] Wild Intelligence Lab. *Detection Of Animals In Drone Images To Support Wildlife Conservation In Namibia*. URL: https://wildintelligencelab.com/detection-of-animals-in-drone-images/ (visited on 01/31/2024) (cit. on p. 5).

[15] Video LAN. *VideoLAN-Homepage*. URL: https://www.videolan.org/ (visited on 01/31/2024) (cit. on p. 6).

[16] Haotian Liu et al. *LLaVA-Homepage*. 2023. URL: https://llava-vl.github.io/ (visited on 01/31/2024) (cit. on p. 10).

[17] Peter Loshin. *What is SSH?* Sept. 2021. URL: https://www.techtarget.com/searchsecurity/definition/Secure-Shell (visited on 01/31/2024) (cit. on p. 10).

[18] luca$_g$97. *Own Roboflow Model*. 2024. URL: https://universe.roboflow.com/interactive-media/semester-project-yzmhy (visited on 01/31/2024) (cit. on p. 11).

[19] lucag97. *Custom Colab Server*. 2024. URL: https://hub.docker.com/r/lucag97/colab-server (visited on 01/31/2024) (cit. on p. 11).

[20] Meta. *Meta Homepage Austria*. 2024. URL: https://www.meta.com/at/en/ (visited on 01/31/2024) (cit. on pp. 8, 9).

[21] Meta. *Segment Anything*. 2023. URL: https://segment-anything.com/ (visited on 01/31/2024) (cit. on p. 9).

[22] PawelPeczek-Roboflow. *Roboflow-YoloV8*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/train-yolov8-object-detection-on-custom-dataset.ipynb (visited on 01/31/2024) (cit. on p. 9).

[23] Roboflow. *Roboflow-Set-Of-Marks*. URL: https://huggingface.co/spaces/Roboflow/SoM (visited on 01/31/2024) (cit. on p. 16).

[24] SkalskiP. *Lists of Roboflow-Models*. URL: https://github.com/roboflow/notebooks (visited on 01/31/2024) (cit. on p. 8).

[25] SkalskiP. *Roboflow-Clip*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/how-to-use-openai-clip-classification.ipynb (visited on 01/31/2024) (cit. on p. 9).

[26] SkalskiP. *Roboflow-YoloNAS*. 2024. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/train-yolo-nas-on-custom-dataset.ipynb (visited on 01/31/2024) (cit. on pp. 8, 9).

[27] SkalskiP. *Roboflow-YoloX*. URL: https://github.com/roboflow/notebooks/blob/main/notebooks/train-yolox-object-detection-on-custom-data.ipynb (visited on 01/31/2024) (cit. on p. 9).

[28] BAMBI Team. *Goal of BAMBI*. URL: https://www.bambi.eco (visited on 01/31/2024) (cit. on p. 4).

[29] yatengLG. *Lists of Models*. URL: https://github.com/Hedlen/awesome-segment-anything/blob/main/README.md (visited on 01/31/2024) (cit. on p. 8).

[30] younesbelkada. *HuggingFace-SAM with Transformers*. URL: https://github.com/huggingface/notebooks/blob/main/examples/segment_anything.ipynb (visited on 01/31/2024) (cit. on p. 10).